

Applied MLOps for Data Engineers: Lab Guide

This document contains the step-by-step instructions, code files, and execution commands for Lab 1 (Tracking) and Lab 2 (Deployment) of the MLOps curriculum.

Lab 1: Experiment Tracking with MLflow

Objective: Track XGBoost experiments predicting California housing prices using a local MLflow server backed by a SQLite database.

Step 1: Initialize the Environment

Create a fresh virtual environment and install the required dependencies for tracking and modeling. Run this in your terminal:

```
python3 -m venv mlops-env
# On Windows use: mlops-env\Scripts\activate
source mlops-env/bin/activate
pip install mlflow xgboost scikit-learn pandas numpy
```

Step 2: Start the MLflow Tracking Server

Launch the local MLflow UI backed by a SQLite database. The database is mandatory to enable the Model Registry feature locally. Run this in a separate terminal window:

```
# Activate the environment in this new terminal first
source mlops-env/bin/activate

# Start the server on port 5000
mlflow server --backend-store-uri sqlite:///mlruns.db --default-artifact-root
./artifacts --host 127.0.0.1 --port 5000
```

Step 3: The Training Script (train.py)

Save the following code as train.py in your working directory. This script handles data fetching, deterministic splitting, and iterative logging to the tracking server.

```
import mlflow
import mlflow.xgboost
import xgboost as xgb
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import numpy as np

def prepare_data():
    print("Fetching and splitting California Housing data...")
```

```

california = fetch_california_housing()
X, y = california.data, california.target
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)
return X_train, X_val, y_train, y_val

def execute_run(run_name, max_depth, learning_rate, X_train, X_val, y_train,
y_val):
    with mlflow.start_run(run_name=run_name):
        params = {
            "max_depth": max_depth,
            "learning_rate": learning_rate,
            "objective": "reg:squarederror",
            "eval_metric": "rmse"
        }
        mlflow.log_params(params)

        dtrain = xgb.DMatrix(X_train, label=y_train)
        dval = xgb.DMatrix(X_val, label=y_val)

        model = xgb.train(
            params=params,
            dtrain=dtrain,
            num_boost_round=100,
            evals=[(dtrain, "train"), (dval, "val")],
            verbose_eval=False
        )

        preds = model.predict(dval)
        rmse = np.sqrt(mean_squared_error(y_val, preds))

        mlflow.log_metric("val_rmse", rmse)
        mlflow.xgboost.log_model(model, artifact_path="xgboost-model")
        print(f"{run_name} Complete | Val RMSE: {rmse:.4f}")

def main():
    mlflow.set_tracking_uri("http://127.0.0.1:5000")
    mlflow.set_experiment("California_Housing_Optimization")

    X_train, X_val, y_train, y_val = prepare_data()

    run_configs = [
        {"name": "Run 1", "depth": 3, "lr": 0.1},
        {"name": "Run 2", "depth": 5, "lr": 0.05},
        {"name": "Run 3", "depth": 7, "lr": 0.01},
    ]

    for config in run_configs:
        execute_run(config["name"], config["depth"], config["lr"], X_train,
X_val, y_train, y_val)

if __name__ == "__main__":
    main()

```

Run the script in your terminal:

```
python train.py
```

Step 4: Comparing and Promoting in the UI

1. 1. Open your browser to <http://127.0.0.1:5000>.
2. 2. Compare Run 1, 2, and 3 in the California_Housing_Optimization experiment.
3. 3. Select Run 2, go to artifacts, and click "Register Model" as "California_Housing_Predictor".
4. 4. Go to the Models tab, select Version 2, and register the model , then promote it and add the alias "staging" to the model.

Lab 2: Deployment with Docker and Kubernetes

Objective: Package the trained XGBoost model into a FastAPI application, containerize it, and orchestrate it locally using Minikube on Windows.

Step 1: Exporting Your Model Artifact

Run this quick Python script to download your staging model artifacts locally into a folder named "model".

```
import mlflow

mlflow.set_tracking_uri("http://127.0.0.1:5000")
model_uri = "models:/California_Housing_Predictor@staging"

# This creates a ./model folder with MLmodel, model.xgb, etc.
mlflow.artifacts.download_artifacts(artifact_uri=model_uri, dst_path="./model")
print("Model downloaded successfully!")
```

Step 2: FastAPI Serving Code (app.py)

Create app.py. This script loads the model into memory at startup and exposes an HTTP /predict endpoint.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import mlflow.xgboost
import xgboost as xgb
import pandas as pd
import os

app = FastAPI(title="California Housing API")
model = None

class HousingInput(BaseModel):
    MedInc: float
    HouseAge: float
    AveRooms: float
    AveBedrms: float
    Population: float
    AveOccup: float
    Latitude: float
    Longitude: float

@app.on_event("startup")
def load_model():
    global model
    model_path = os.getenv("MODEL_PATH", "./model")
    model = mlflow.xgboost.load_model(model_path)
    print("Model loaded successfully!")

@app.get("/health")
def health_check():
    return {"status": "healthy", "model_loaded": model is not None}
```

```
@app.post("/predict")
def predict(input_data: HousingInput):
    if model is None:
        raise HTTPException(status_code=500, detail="Model not loaded")
    try:
        input_df = pd.DataFrame([input_data.dict()])
        dmatrix_input = xgb.DMatrix(input_df)

        raw_prediction = float(model.predict(dmatrix_input)[0])
        actual_price = raw_prediction * 100000.0

        return {"prediction": f"${actual_price:,.2f}"}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

Step 3: Dependencies (requirements.txt)

Create requirements.txt to pin library versions:

```
fastapi==0.110.0
uvicorn==0.28.0
mlflow==2.11.1
xgboost==2.0.3
pandas==2.2.1
numpy==1.26.4
pydantic==2.6.4
scikit-learn==1.4.2
```

Step 4: Containerization (Dockerfile)

Create a .dockerignore file containing: venv/, mlruns/, mlruns.db. Then create the Dockerfile:

```
FROM python:3.10-slim
WORKDIR /app
RUN apt-get update && apt-get install -y build-essential && rm -rf
/var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
COPY ./model /app/model
ENV MODEL_PATH=/app/model
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 5: Kubernetes Manifest (k8s-deployment.yaml)

Create k8s-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: housing-api-deployment
spec:
  replicas: 2
  selector:
```

```

    matchLabels:
      app: housing-api
  template:
    metadata:
      labels:
        app: housing-api
    spec:
      containers:
      - name: housing-api
        image: california-housing-api:v1
        imagePullPolicy: Never
        ports:
        - containerPort: 8000
        readinessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 5
          periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: housing-api-service
spec:
  type: LoadBalancer
  selector:
    app: housing-api
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8000

```

Step 6: Build and Deploy to Minikube

Run the following commands in an Administrator PowerShell to build the image inside Minikube and deploy it:

Install minikube on windows if it's not there

```
winget install Kubernetes.Minikube
```

```
minikube dashboard
```

```
# 1. Start Minikube (if not running)
```

```
minikube start --driver=docker
```

```
# 2. Build image directly in Minikube registry
```

```
minikube image build -t california-housing-api:v1 .
```

```
# 3. Apply the Kubernetes resources
```

```
kubectl apply -f k8s-deployment.yaml
```

```
, this works also in case of restart

kubectl rollout restart deployment housing-api-deployment

kubectl get pods

# 4. Check pod status (wait for 1/1 READY)
kubectl get pods
```

Step 7: Testing the Deployment

Set up port-forwarding in one terminal window to expose the service to your local machine:

```
kubectl port-forward svc/housing-api-service 8080:80
```

➔ use postman to post request

```
curl -X 'POST' \
  'http://127.0.0.1:8080/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "MedInc": 8.3252,
    "HouseAge": 41.0,
    "AveRooms": 6.9841,
    "AveBedrms": 1.0238,
    "Population": 322.0,
    "AveOccup": 2.5556,
    "Latitude": 37.88,
    "Longitude": -122.23
  }'
```